Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

ЗВІТ

з лабораторної роботи № 2
«Розроблення підсистеми розмежування доступу»
з дисципліни «Безпека програмного забезпечення»

Варіант 12

Виконав: здобувач групи ІП-33
Ломакін Максим

Перевірив:
Курченко Олег Анастасійович

Київ — 2026

1 МЕТА ТА ЗАВДАННЯ РОБОТИ

Мета роботи – набути практичних навичок побудови та перевірки диспетчера доступу (reference monitor), що реалізує задану модель розмежування доступу, веде журнали своїх рішень і виявляє аномалії в поведінці суб'єктів.

Завдання: реалізувати диспетчер доступу за моделлю варіанта як підклас `Policy`; наповнити політику та застосувати додаткове правило; перевірити три властивості еталонного монітора; реалізувати два журнали й детектор двох аномалій; написати власні модульні тести; продемонструвати запити «хто має доступ до об'єкта» та «до чого має доступ суб'єкт».

Варіант 12. Модель – ABAC; додаткове правило – заборонні записи (deny) з пріоритетом над дозвільними; аномалії – послідовний перебір ідентифікаторів об'єктів (A3) і сплеск відмов у доступі від одного суб'єкта (A1); суб'єктів – 8, об'єктів – 21. Вивід команди `python3 -m common.variants 12 --lab 2`:

```
Варіант 12

  LAB2
    model: ABAC
    extra_rule: заборонні записи (deny) з пріоритетом над дозвільними
    anomalies:
      - послідовний перебір ідентифікаторів об'єктів
      - сплеск відмов у доступі від одного суб'єкта
    subjects: 8
    objects: 21
```

Каркас і навчальний приклад із репозиторію курсу [5] працюють: команда `PYTHONPATH=. pytest tests/test_example.py -q` завершується рядком «10 passed in 0.02s».

2 ТЕОРЕТИЧНІ ВІДОМОСТІ

Диспетчер доступу (reference monitor) – посередник, через якого проходить кожне звернення суб'єкта до об'єкта [2]. Він має задовольняти три властивості: повне посередництво, захищеність і верифіковність [1]. Із цих властивостей випливають рішення, прийняті в роботі: політику зосереджено в одному класі з єдиним методом `evaluate`, прикладна підсистема отримує лише монітор, а не політику, і рішення за відсутності явного дозволу завжди є відмовою – принцип безпечних значень за замовчуванням [1].

Варіант задає атрибутну модель ABAC [3]: права не призначаються парам «суб'єкт – об'єкт», а обчислюються правилами-предикатами над атрибутами

суб'єкта й об'єкта. Тому замість матриці доступу політика зберігає атрибути та перелік правил, а множина наданих прав perm(s, o) є об'єднанням прав усіх правил, предикати яких істинні для пари (s, o). Така модель обрана варіантом і добре пасує до документів організації: доступ визначають підрозділ, посада, рівень допуску та гриф, а не індивідуальні призначення.

Додаткове правило – заборонні записи з пріоритетом над дозвільними. Рішення описує формула allow(s, o, a) = (a ⊆ perm(s, o)) ∧ (a ∩ deny(s, o) = ∅): дія дозволена, лише якщо всі запитані права надано і жодне з них не заборонено. У коді пріоритет заборони забезпечено порядком перевірок – заборонні правила переглядаються першими, тож жоден дозвіл не може їх перекрити. Заборони в ABAC теж задано предикатами над атрибутами.

Реєстраційний журнал фіксує кожне рішення монітора, операційний – лише виконані зміни стану [4]. За реєстраційним журналом детектор шукає аномалії. Його якість визначають хибнопозитивні й хибнонегативні спрацювання, тому пороги зроблено параметрами, а для сплеску відмов додано ковзне часове вікно: кількість відмов без урахування часу не відрізняє атаку від помилок, накопичених за робочий день.

3 ОПИС РОЗРОБЛЕНОГО РІШЕННЯ

3.1 Структурна схема підсистеми

Підсистема складається з п'яти частин (рисунок 3.1). Суб'єкти працюють з документами лише через сховище `DocumentStore`. Кожна його операція спершу викликає `ReferenceMonitor.enforce`; монітор звертається до політики `VariantPolicy`, отримує рішення `Decision` і записує його в реєстраційний журнал `AuditLog`. Якщо доступ дозволено, сховище виконує операцію, а зміну стану фіксує в операційному журналі `OperationLog`. Детектор `VariantDetector` читає реєстраційний журнал і формує сповіщення для адміністратора безпеки.

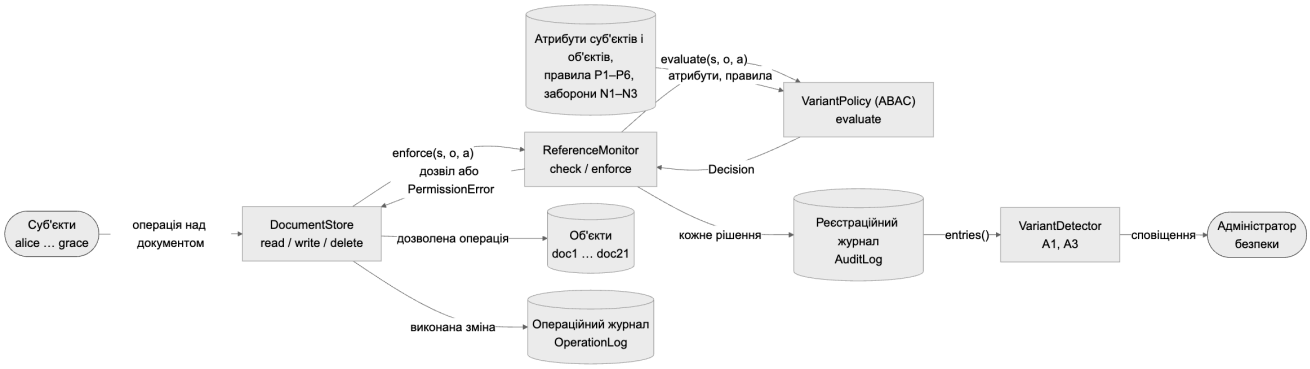


Рисунок 3.1 – Структурна схема підсистеми розмежування доступу

Каркас `accesscontrol/monitor.py` (класи `Permission`, `Decision`, `Policy`, `AuditLog`, `ReferenceMonitor`, `AnomalyDetector`, функція `subjects_with_access`) узято з репозиторію курсу [5] без змін. Власний код розміщено в чотирьох модулях (таблиця 3.1).

Таблиця 3.1 – Власні модулі підсистеми

Файл	Вміст
`mypolicy.py`	політика `VariantPolicy` (ABAC із заборонами), наповнення `build_policy`, запит `objects_accessible_by`
`detector.py`	детектор `VariantDetector` для аномалій A1 і A3
`storage.py`	сховище `DocumentStore`, операційний журнал `OperationLog`, журнал із заданим джерелом часу `TimedAuditLog`
`demo.py`	демонстрація роботи підсистеми
`tests/test_variant.py`	22 модульні тести

3.2 Політика за моделлю ABAC

Політика зберігає атрибути суб'єктів і об'єктів та перелік правил. Правило `Rule` – це код, назва, набір прав і предикат над атрибутами суб'єкта й об'єкта. Дозвільні правила зберігаються окремо від заборонних. Атрибути середовища (час, мережа) в цій політиці не використовуються. Рішення обчислює метод `evaluate`:

```
def evaluate(self, subject: str, obj: str, action: Permission) -> Decision:
    s = self._subjects.get(subject)
    o = self._objects.get(obj)
    if s is None:
        return Decision(False, "невідомий суб'єкт (safe default)")
    if o is None:
        return Decision(False, "невідомий об'єкт (safe default)")
    if action == Permission.NONE:
        return Decision(False, "порожня дія (safe default)")

    for rule in self._deny:
        if action & rule.permissions and rule.condition(s, o):
            return Decision(False, f"заборона {rule.code}: {rule.title}")
```

```
effective = Permission.NONE
used: list[str] = []
for rule in self._allow:
    if rule.condition(s, o):
        effective |= rule.permissions
        if action & rule.permissions:
            used.append(rule.code)
if action & effective == action:
    return Decision(True, "надано: " + ", ".join(used))
return Decision(False, "явного дозволу немає (safe default)")
```

Спочатку відхиляються запити з невідомим суб'єктом, невідомим об'єктом або порожньою дією. Далі перебираються заборонні правила: якщо запитана дія перетинається з правами правила і його предикат істинний, повертається відмова з кодом заборони, і до дозвільних правил справа не доходить. Потім ефективні права обчислюються як об'єднання прав усіх дозвільних правил з істинним предикатом; доступ надається, лише якщо вони містять усі запитані права (`action & effective == action`). В іншому разі останній рядок методу повертає відмову – це правило безпечних значень за замовчуванням.

3.3 Наповнення політики та заборонні записи

Політика описує вигадану компанію з трьома підрозділами (розробка, фінанси, кадри) та аудитором; усі назви умовні. Атрибути суб'єктів наведено в таблиці 3.2, об'єктів – у таблиці 3.3, правила – у таблиці 3.4.

Таблиця 3.2 – Суб'єкти та їхні атрибути

Суб'єкт	Підрозділ	Посада	Допуск	Стан
alice	dev	engineer	2	active
bob	dev	lead	3	active
heidi	dev	intern	1	active
carol	finance	accountant	2	active
dave	finance	manager	3	active
erin	hr	specialist	2	suspended
frank	hr	manager	3	active
grace	audit	auditor	3	active

Таблиця 3.3 – Об'єкти та їхні атрибути

Об'єкт	Підрозділ	Тип	Гриф	Власник	Юридична фіксація
doc1	dev	spec	1	alice	–
doc2	dev	spec	2	alice	–
doc3	dev	design	2	bob	так
doc4	dev	design	3	bob	–
doc5	dev	report	2	alice	–

Об'єкт	Підрозділ	Тип	Гриф	Власник	Юридична фіксація
doc6	dev	guide	1	heidi	–
doc7	dev	roadmap	3	bob	–
doc8	finance	report	1	carol	–
doc9	finance	invoice	2	carol	–
doc10	finance	invoice	2	carol	–
doc11	finance	budget	3	dave	–
doc12	finance	budget	3	dave	так
doc13	finance	report	2	dave	–
doc14	finance	payroll	3	dave	–
doc15	hr	policy	1	frank	–
doc16	hr	policy	2	erin	–
doc17	hr	personal	3	frank	–
doc18	hr	personal	3	frank	–
doc19	hr	contract	2	erin	–
doc20	hr	vacancy	1	erin	–
doc21	hr	personal	3	frank	–

Таблиця 3.4 – Правила політики

Код	Права	Умова
P1	READ	підрозділ суб'єкта збігається з підрозділом об'єкта і допуск не нижчий за гриф
P2	WRITE	те саме, що P1, і посада суб'єкта – не intern
P3	DELETE	той самий підрозділ і посада lead або manager
P4	READ | WRITE	суб'єкт є власником об'єкта
P5	READ	гриф об'єкта дорівнює 1 (загальнодоступний документ)
P6	READ	посада суб'єкта – auditor
N1	WRITE | DELETE	заборона: на об'єкт накладено юридичну фіксацію (legal_hold)
N2	ADMIN (усі дії)	заборона: обліковий запис суб'єкта призупинено (status = suspended)
N3	READ	заборона: посада auditor і тип об'єкта personal

Заборони мають різну природу: N1 залежить лише від атрибута об'єкта, N2 – лише від атрибута суб'єкта, N3 – від обох. У коді правила задано предикатами:

```
policy.allow("P1", "читання у своєму підрозділі", Permission.READ,
             lambda s, o: _same_department(s, o) and _cleared(s, o))
policy.allow("P2", "редагування у своєму підрозділі", Permission.WRITE,
             lambda s, o: (_same_department(s, o) and _cleared(s, o)
                           and s["position"] != "intern"))
policy.allow("P3", "керівник вилучає документи підрозділу", Permission.DELETE,
             lambda s, o: (_same_department(s, o)
                           and s["position"] in ("lead", "manager")))
policy.allow("P4", "власник документа", Permission.READ | Permission.WRITE,
             lambda s, o: o["owner"] == s["id"])
policy.allow("P5", "загальнодоступний документ", Permission.READ,
             lambda s, o: o["classification"] == 1)
policy.allow("P6", "аудитор читає документи", Permission.READ,
             lambda s, o: s["position"] == "auditor")
```

```
if with_deny:
    policy.deny("N1", "юридична фіксація документа",
                Permission.WRITE | Permission.DELETE,
                lambda s, o: bool(o["legal_hold"]))
    policy.deny("N2", "обліковий запис призупинено", Permission.ADMIN,
                lambda s, o: s["status"] == "suspended")
    policy.deny("N3", "аудитор не читає персональні дані", Permission.READ,
                lambda s, o: (s["position"] == "auditor"
                              and o["kind"] == "personal"))
```

Після наповнення `build_policy` викликає `seal()`, і подальші зміни політики стають неможливими. Параметр `with_deny=False` будує ту саму політику без заборон – базову модель, з якою порівнюються рішення.

3.4 Детектор аномалій

Клас `VariantDetector` аналізує `log.entries()`. Пороги задаються параметрами конструктора: `deny_threshold` – кількість відмов, `window_seconds` – ширина вікна, `run_threshold` – довжина серії.

A1, сплеск відмов. Для кожного суб'єкта збираються моменти відмов (поле `ts`); ковзним вікном обчислюється найбільша кількість відмов за `window_seconds` секунд і порівнюється з порогом.

```
def _deny_bursts(self, entries: list[dict]) -> list[str]:
    denials: dict[str, list[float]] = collections.defaultdict(list)
    for entry in entries:
        if not entry["allowed"]:
            denials[entry["subject"]].append(entry["ts"])

    alerts = []
    for subject, moments in denials.items():
        moments.sort()
        peak, start = 0, 0
        for end, moment in enumerate(moments):
            while moment - moments[start] > self.window_seconds:
                start += 1
            peak = max(peak, end - start + 1)
        if peak >= self.deny_threshold:
            alerts.append(
                f"A1: сплеск відмов — {subject}: {peak} відмов "
                f"за {self.window_seconds} с (поріг {self.deny_threshold})")
    return alerts
```

A3, послідовний перебір ідентифікаторів. Для кожного суб'єкта будується хронологічний перелік запитаних об'єктів (повторне звернення до того самого об'єкта поспіль не враховується). З назви об'єкта береться числовий суфікс, і шукається найдовша серія, у якій номер щоразу змінюється рівно на одиницю в один бік. Враховуються і дозволені, і відхилені звернення, бо перебір небезпечний незалежно від результату.

```
def _enumerations(self, entries: list[dict]) -> list[str]:
    requested: dict[str, list[str]] = collections.defaultdict(list)
    for entry in entries:
        objects = requested[entry["subject"]]
        if not objects or objects[-1] != entry["object"]:
            objects.append(entry["object"])

    alerts = []
    for subject, objects in requested.items():
        first, last = self._longest_run(objects)
        length = last - first + 1
        if length >= self.run_threshold:
            alerts.append(
                f"A3: послідовний перебір — {subject}: {length} об'єктів поспіль "
                f"({objects[first]}…{objects[last]}, поріг {self.run_threshold})")
    return alerts

@staticmethod
def _longest_run(objects: list[str]) -> tuple[int, int]:
    """Межі найдовшої серії, де номер об'єкта щоразу змінюється на 1 в один
бік."""
    numbers = [_object_number(name) for name in objects]
    best = (0, -1)
    start, step = 0, 0
    for i, number in enumerate(numbers):
        previous = numbers[i - 1] if i else None
        known = number is not None and previous is not None
        delta = number - previous if known else None
        if delta not in (1, -1):
            start, step = i, 0
        elif delta != step:
            start, step = i - 1, delta
        if number is not None and i - start > best[1] - best[0]:
            best = (start, i)
    return best
```

4 РЕЗУЛЬТАТИ

4.1 Очікувані та фактичні рішення

Таблиця 4.1 містить очікувані й фактичні рішення `evaluate` для 20 показових запитів; усі фактичні рішення збігаються з очікуваними. Стовпець «Без заборон» показує рішення базової моделі. Виділено 6 рядків, у яких заборонний запис перекрив наданий дозвіл.

Таблиця 4.1 – Очікувані та фактичні рішення evaluate

Суб'єкт	Об'єкт	Дія	Очікувано	Фактично	Без заборон	Причина рішення
alice	doc2	READ	дозвіл	дозвіл	дозвіл	надано: P1, P4
alice	doc2	READ | WRITE	дозвіл	дозвіл	дозвіл	надано: P1, P2, P4
alice	doc2	ADMIN	відмова	відмова	відмова	явного дозволу немає (safe default)
alice	doc4	READ	відмова	відмова	відмова	явного дозволу немає (safe default)
alice	doc8	READ	дозвіл	дозвіл	дозвіл	надано: P5
alice	doc9	READ	відмова	відмова	відмова	явного дозволу немає (safe default)

Суб'єкт	Об'єкт	Дія	Очікувано	Фактично	Без заборон	Причина рішення
heidi	doc2	READ	відмова	відмова	відмова	явного дозволу немає (safe default)
heidi	doc6	WRITE	дозвіл	дозвіл	дозвіл	надано: P4
bob	doc4	DELETE	дозвіл	дозвіл	дозвіл	надано: P3
bob	doc3	READ	дозвіл	дозвіл	дозвіл	надано: P1, P4
bob	**doc3**	**WRITE**	**відмова**	**відмова**	**дозвіл**	**заборона N1: юридична фіксація документа**
bob	**doc3**	**DELETE**	**відмова**	**відмова**	**дозвіл**	**заборона N1: юридична фіксація документа**
dave	**doc12**	**WRITE**	**відмова**	**відмова**	**дозвіл**	**заборона N1: юридична фіксація документа**
erin	**doc16**	**READ**	**відмова**	**відмова**	**дозвіл**	**заборона N2: обліковий запис призупинено**
erin	**doc15**	**READ**	**відмова**	**відмова**	**дозвіл**	**заборона N2: обліковий запис призупинено**
grace	doc11	READ	дозвіл	дозвіл	дозвіл	надано: P6
grace	doc11	WRITE	відмова	відмова	відмова	явного дозволу немає (safe default)
grace	**doc17**	**READ**	**відмова**	**відмова**	**дозвіл**	**заборона N3: аудитор не читає персональні дані**
mallory	doc1	READ	відмова	відмова	відмова	невідомий суб'єкт (safe default)
alice	doc99	READ	відмова	відмова	відмова	невідомий об'єкт (safe default)

4.2 Три властивості диспетчера та журнали

Політику обгорнуто монітором, а монітор передано сховищу документів. Операція сховища виконується лише після успішного `enforce`:

```
def write(self, subject: str, obj: str, text: str) -> None:
    self._monitor.enforce(subject, obj, Permission.WRITE)
    self._documents[obj] = text
    self.oplog.record(subject, obj, "WRITE")
```

Повне посередництво. `DocumentStore` не має посилання на політику – лише на монітор, а `evaluate` під час обслуговування звернень викликається тільки з `ReferenceMonitor.check`. Серія з 14 звернень дала рівно 14 записів у реєстраційному журналі (8 дозволів, 6 відмов): жодне звернення не оминуло монітор.

Захищеність. Внутрішні структури політики приховано: атрибути й переліки суб'єктів та об'єктів доступні лише як незмінні подання (`MappingProxyType`), правила – як кортеж-копія, а після `seal()` методи зміни політики підіймають виняток (тест `test_sealed_policy_rejects_changes`). Обмеження: Python не забороняє звернення до атрибутів із підкресленням, а журнал каркаса не захищено від підробки, тому в реальній системі монітор, політику й журнал слід винести в окремий процес.

Верифіковність. Усю логіку рішення зосереджено в одному методі `evaluate` на 22 рядки; правила є чистими предикатами без побічних ефектів, тому кожне перевіряється окремо. Поведінку покрито 22 тестами.

Вивід серії звернень і реєстраційний журнал `AuditLog.dump()` (час – у секундах від початку епохи, перший запис відповідає 09:05 12.10.2026):

```
відмову перехоплено: alice -> doc4 [read]: явного дозволу немає (safe default)
відмову перехоплено: heidi -> doc1 [write]: явного дозволу немає (safe default)
відмову перехоплено: bob -> doc3 [delete]: заборона N1: юридична фіксація
документа
відмову перехоплено: dave -> doc12 [write]: заборона N1: юридична фіксація
документа
відмову перехоплено: erin -> doc16 [read]: заборона N2: обліковий запис
призупинено
відмову перехоплено: grace -> doc17 [read]: заборона N3: аудитор не читає
персональні дані
звернень до монітора: 14, записів у реєстраційному журналі: 14
дозволів: 8, відмов: 6
```

```
{"ts": 1791788700.0, "subject": "alice", "object": "doc2", "action": "READ",
"allowed": true, "reason": "надано: P1, P4"}
{"ts": 1791789000.0, "subject": "alice", "object": "doc2", "action": "WRITE",
"allowed": true, "reason": "надано: P2, P4"}
{"ts": 1791789300.0, "subject": "alice", "object": "doc4", "action": "READ",
"allowed": false, "reason": "явного дозволу немає (safe default)"}
{"ts": 1791789600.0, "subject": "heidi", "object": "doc6", "action": "READ",
"allowed": true, "reason": "надано: P1, P4, P5"}
{"ts": 1791789900.0, "subject": "heidi", "object": "doc1", "action": "WRITE",
"allowed": false, "reason": "явного дозволу немає (safe default)"}
{"ts": 1791790200.0, "subject": "bob", "object": "doc3", "action": "DELETE",
"allowed": false, "reason": "заборона N1: юридична фіксація документа"}
{"ts": 1791790500.0, "subject": "bob", "object": "doc5", "action": "DELETE",
"allowed": true, "reason": "надано: P3"}
{"ts": 1791790800.0, "subject": "carol", "object": "doc9", "action": "READ",
"allowed": true, "reason": "надано: P1, P4"}
{"ts": 1791791100.0, "subject": "dave", "object": "doc12", "action": "WRITE",
"allowed": false, "reason": "заборона N1: юридична фіксація документа"}
{"ts": 1791791400.0, "subject": "erin", "object": "doc16", "action": "READ",
"allowed": false, "reason": "заборона N2: обліковий запис призупинено"}
{"ts": 1791791700.0, "subject": "grace", "object": "doc11", "action": "READ",
"allowed": true, "reason": "надано: P6"}
{"ts": 1791792000.0, "subject": "grace", "object": "doc17", "action": "READ",
"allowed": false, "reason": "заборона N3: аудитор не читає персональні дані"}
{"ts": 1791792300.0, "subject": "frank", "object": "doc15", "action": "WRITE",
"allowed": true, "reason": "надано: P2, P4"}
{"ts": 1791792600.0, "subject": "dave", "object": "doc14", "action": "READ",
"allowed": true, "reason": "надано: P1, P4"}
```

Операційний журнал містить лише 3 виконані зміни стану:

```
{"ts": 1791789000.0, "subject": "alice", "object": "doc2", "operation": "WRITE"}
{"ts": 1791790500.0, "subject": "bob", "object": "doc5", "operation": "DELETE"}
{"ts": 1791792300.0, "subject": "frank", "object": "doc15", "operation": "WRITE"}
```

4.3 Спрацювання детектора

На журналі звичайної роботи детектор не дає сповіщень. Після шести відхилених звернень призупиненого користувача erin за 12 с і послідовного

читання аудитором grace документів doc1–doc8 він повертає два сповіщення. Останні три рядки показують чутливість до порога серії.

```
звичайна робота: []
після сплеску відмов erin і перебору grace:
  A1: сплеск відмов — erin: 6 відмов за 60 с (поріг 5)
  A3: послідовний перебір — grace: 8 об'єктів поспіль (doc1…doc8, поріг 5)
-- чутливість до порога серії --
alice переглядає doc1–doc5, поріг 5: ["A3: послідовний перебір — alice: 5 об'єктів
поспіль (doc1…doc5, поріг 5)"]
alice переглядає doc1–doc5, поріг 8: []
журнал із перебором grace, поріг 8: ["A3: послідовний перебір — grace: 8 об'єктів
поспіль (doc1…doc8, поріг 8)"]
```

4.4 Запити «хто має доступ» і «до чого має доступ»

Запит «хто має доступ до об'єкта» виконує функція каркаса `subjects_with_access`, двоїстий запит реалізовано функцією `objects_accessible_by`:

```
def objects_accessible_by(policy: Policy, subject: str, objects: Iterable[str],
                          action: Permission) -> list[str]:
    return [o for o in objects if policy.evaluate(subject, o, action).allowed]
```

```
хто має READ до doc1: ['alice', 'bob', 'heidi', 'carol', 'dave', 'frank', 'grace']
хто має WRITE до doc3: []
хто має DELETE до doc4: ['bob']
хто має READ до doc17: ['frank']
до чого alice має READ: ['doc1', 'doc2', 'doc3', 'doc5', 'doc6', 'doc8', 'doc15',
'doc20']
до чого alice має WRITE: ['doc1', 'doc2', 'doc5', 'doc6']
до чого heidi має READ: ['doc1', 'doc6', 'doc8', 'doc15', 'doc20']
до чого erin має READ: []
до чого grace має READ: ['doc1', 'doc2', 'doc3', 'doc4', 'doc5', 'doc6', 'doc7',
'doc8', 'doc9', 'doc10', 'doc11', 'doc12', 'doc13', 'doc14', 'doc15', 'doc16',
'doc19', 'doc20']
до чого bob має DELETE: ['doc1', 'doc2', 'doc4', 'doc5', 'doc6', 'doc7']
перевірено трійок «суб'єкт — об'єкт — дія»: 504, розбіжностей між запитами: 0
```

Результати узгоджуються між собою і з таблицею 4.1. Наприклад, bob – єдиний у відповіді на «хто має DELETE до doc4», і doc4 є у відповіді на «до чого bob має DELETE»; doc3 у цьому переліку немає, а відповідь на «хто має WRITE до doc3» порожня через заборону N1. Суб'єкт erin не має доступу ні до чого (N2) і не входить до жодної відповіді «хто має доступ». Перевірка всіх 504 трійок «суб'єкт – об'єкт – дія» розбіжностей між двома запитами не виявила.

4.5 Модульні тести

Файл `tests/test_variant.py` містить 22 тести: правила над атрибутами, безпечні значення за замовчуванням, три заборонні записи, монітор і журнали, обидві аномалії (спрацювання та відсутність спрацювання) і узгодженість запитів. Запуск: `PYTHONPATH=. pytest tests/test_variant.py -v`.

```
============================= test session starts ==============================
collecting ... collected 22 items

tests/test_variant.py::test_policy_size_matches_variant PASSED
tests/test_variant.py::test_attribute_rule_grants_access PASSED
tests/test_variant.py::test_clearance_attribute_limits_access PASSED
tests/test_variant.py::test_owner_rule_applies_regardless_of_position PASSED
tests/test_variant.py::test_safe_default_without_matching_rule PASSED
tests/test_variant.py::test_safe_default_for_unknown_subject_and_object PASSED
tests/test_variant.py::test_combined_action_requires_every_permission PASSED
tests/test_variant.py::test_deny_overrides_allow PASSED
tests/test_variant.py::test_deny_for_suspended_subject_covers_all_actions PASSED
tests/test_variant.py::test_deny_on_subject_and_object_attributes PASSED
tests/test_variant.py::test_monitor_records_every_request PASSED
tests/test_variant.py::test_enforce_raises_on_denial PASSED
tests/test_variant.py::test_sealed_policy_rejects_changes PASSED
tests/test_variant.py::test_operation_log_keeps_only_performed_changes PASSED
tests/test_variant.py::test_detector_reports_deny_burst PASSED
tests/test_variant.py::test_detector_ignores_denials_below_threshold_or_outside_wi
ndow PASSED
tests/test_variant.py::test_detector_reports_sequential_enumeration PASSED
tests/test_variant.py::test_detector_reports_descending_enumeration PASSED
tests/test_variant.py::test_detector_ignores_non_sequential_requests PASSED
tests/test_variant.py::test_detector_finds_both_anomalies_and_none_in_clean_log
PASSED
tests/test_variant.py::test_queries_are_consistent PASSED
tests/test_variant.py::test_queries_reflect_deny_rules PASSED

============================== 22 passed in 0.06s ==============================
```

5 АНАЛІЗ ОТРИМАНИХ РЕЗУЛЬТАТІВ

1. Вплив моделі на складність двох запитів. ABAC не зберігає матрицю доступу ні за стовпцями, ні за рядками: у політиці є лише атрибути 8 суб'єктів і 21 об'єктів та 9 правил-предикатів. Тому жоден із двох запитів не виконується «природно», як читання готового списку. Запит «хто має доступ до об'єкта» потребує обчислити `evaluate` для кожного з 8 суб'єктів, запит «до чого має доступ суб'єкт» – для кожного з 21 об'єктів; кожне обчислення перебирає до 9 правил. Обидва запити є повним обходом, і за 21 об'єкта проти 8 суб'єктів другий удвічі-втричі дорожчий. Це плата за гнучкість: щоб додати документ doc22, досить задати його атрибути, і права всіх суб'єктів визначаться самі, тоді як в ACL довелося б заповнювати його список, а в capabilities – видавати квитки кожному суб'єктові. Узгодженість запитів перевірено на всіх 504 трійках «суб'єкт – об'єкт – дія»: розбіжностей немає.

2. Кількість записів журналу та повне посередництво. Серія звичайної роботи складалася з 14 звернень: 13 операцій сховища і один прямий виклик `check`. Реєстраційний журнал містить рівно 14 записів – 8 дозволів і 6 відмов; після сценаріїв з аномаліями (ще 6 і 8 звернень) записів стало 28. Жодне звернення

не загубилося й не продублювалося, бо `DocumentStore` отримує лише монітор і викликає `enforce` першим рядком кожної операції, а `check` – єдине місце, де викликається `evaluate`, і він записує рішення до повернення результату. Операційний журнал містить лише 3 записи – виконані зміни стану (два записи WRITE і одне вилучення): три відхилені спроби зміни та всі читання до нього не потрапили. Виняток становлять аналітичні запити «хто має доступ» і «до чого має доступ»: вони викликають `evaluate` напряму й у журналі не відображаються, тому їх слід надавати лише адміністраторові безпеки, а не прикладним підсистемам.

3. Вплив додаткового правила. Заборонні записи перевіряються до дозвільних правил, тому змінюють рішення лише в бік відмови. Із 504 трійок базова модель (без заборон) дозволяє 136, а з заборонами – 117: рішення змінилося для 19 трійок (N1 – 5, N2 – 11, N3 – 3). Показова пара – bob і doc3: bob є власником документа (P4) і керівником підрозділу (P3), тож базова модель дозволяє йому і WRITE, і DELETE; після накладення юридичної фіксації N1 обидві дії відхиляються з причиною «заборона N1», а READ лишається дозволеним, бо N1 його не охоплює. Друга пара – erin і doc16: вона власниця документа, але її обліковий запис призупинено, і N2 відхиляє навіть читання загальнодоступного doc15. Третя – grace і doc17: правило P6 дає аудиторові читання всіх документів, а N3 вилучає з них персональні дані. У таблиці рішень правило змінило 6 рядків із 20.

4. Умови спрацювання детекторів і вибір порогів. Детектор A1 спрацьовує, коли один суб'єкт отримав не менше 5 відмов у межах ковзного вікна 60 с: шість відмов erin за 12 с дали сповіщення. Він мовчить, якщо відмов чотири, якщо шість відмов розтягнуто з кроком 30 с (у вікно потрапляє щонайбільше три) і на журналі звичайної роботи, де 6 відмов належать шістьом різним суб'єктам з інтервалом 5 хв. Поріг 5 обрано тому, що випадкова помилка користувача дає одну–три відмови поспіль, а п'ять за хвилину вже свідчать про автоматичні спроби. Детектор A3 спрацьовує, коли номери об'єктів у послідовних зверненнях одного суб'єкта щонайменше 5 разів поспіль змінюються рівно на одиницю в один бік, незалежно від того, дозволено звернення чи ні: читання grace doc1…doc8 дало сповіщення, хоча всі вісім звернень дозволені. Він мовчить, якщо серію розірвано (порядок 1, 2, 3, 9, 4, 5, 6, 13, 7, 8 дає найдовшу серію 3). Поріг 5 узгоджено з розміром політики: у підрозділі сім документів, і користувач зазвичай відкриває

два-три сусідні. Обидва пороги та ширина вікна є параметрами конструктора, тож їх можна калібрувати без зміни коду.

5. Хибні спрацювання. Можливі обидва типи помилок. Хибнопозитивне спрацювання A3: alice за порядком переглядає документи власного підрозділу doc1–doc5 з інтервалом 3 хв – це легітимна робота, але за порога 5 детектор видає сповіщення. Якщо підняти поріг до 8, сповіщення зникає, а перебір grace (вісім об'єктів) ще виявляється; проте тоді перебір семи документів одного підрозділу буде пропущено. Хибнопозитивне спрацювання A1: клієнтська програма призупиненого користувача п'ять разів автоматично повторює той самий запит. Хибнонегативні: порушник перебирає документи з кроком 2 (doc1, doc3, doc5, …) або у випадковому порядку – A3 мовчить; робить не більше чотирьох заборонених звернень на хвилину – мовчить A1. Крім того, A3 не враховує час, тому повільний перебір він виявить, але й звичайний послідовний перегляд, розтягнутий на тиждень, теж позначить як аномалію. Отже, сповіщення детектора є приводом для перевірки, а не доказом.

ВИСНОВКИ

1. Реалізовано диспетчер доступу за моделлю ABAC із заборонними записами: клас `VariantPolicy` обчислює рішення за 9 правилами над атрибутами 8 суб'єктів і 21 об'єктів; із 504 можливих трійок «суб'єкт – об'єкт – дія» дозволено 117, решту відхилено, зокрема за правилом безпечних значень за замовчуванням.
2. Заборонні записи мають пріоритет над дозволами: три заборони N1–N3 змінили рішення для 19 трійок, і в усіх випадках дозвіл базової моделі став відмовою; зворотна зміна неможлива, бо заборони перевіряються першими.
3. Повне посередництво підтверджено збігом кількості звернень і записів реєстраційного журналу (14 і 14); операційний журнал містить лише 3 виконані зміни стану. Захищеність політики забезпечено її запечатуванням і незмінними поданнями атрибутів, верифіковність – тим, що метод `evaluate` займає 22 рядки коду. Журнал каркаса від підробки не захищено.
4. Детектор виявляє обидві аномалії варіанта: сплеск відмов (6 відмов erin за 12 с за порога 5 у вікні 60 с) і послідовний перебір (doc1…doc8 від grace за порога серії 5), а на журналі звичайної роботи не дає жодного сповіщення.

За порога серії 5 можливе хибнопозитивне спрацювання на перегляді doc1–doc5, за порога 8 воно зникає.

5. Написано 22 власні модульні тести, усі проходять (разом із 10 тестами навчального прикладу – 32); вони покривають правила над атрибутами, безпечні значення за замовчуванням, кожну із заборон, монітор, обидва журнали, обидві аномалії та узгодженість запитів «хто має доступ» і «до чого має доступ».

ВИКОРИСТАННЯ ГЕНЕРАТИВНИХ ІНСТРУМЕНТІВ

Генеративний інструмент ШІ використовувався як допоміжний засіб для добору методів, підготовки чернеток та тексту звіту. Усі наведені результати отримано запуском коду; параметри варіанта, оцінки й вивід програм відтворюються командами з репозиторію.

ПЕРЕЛІК ПОСИЛАНЬ

1. Saltzer J. H. The Protection of Information in Computer Systems / J. H. Saltzer, M. D. Schroeder // Proceedings of the IEEE. – 1975. – Vol. 63, № 9. – P. 1278–1308.
2. Anderson J. P. Computer Security Technology Planning Study / J. P. Anderson. – Bedford : ESD-TR-73-51, USAF Electronic Systems Division, 1972. – Vol. II.
3. NIST SP 800-162. Guide to Attribute Based Access Control (ABAC) Definition and Considerations / V. Hu [et al.]. – Gaithersburg : NIST, 2014.
4. Bishop M. Computer Security: Art and Science / M. Bishop. – 2nd ed. – Boston : Addison-Wesley, 2018. – 1440 p.
5. Соколовський В. В. Навчально-методичний комплекс дисципліни «Безпека програмного забезпечення»: код лабораторних робіт [Електронний ресурс]. – Режим доступу: https://github.com/sokolusha/software-security-course

ДОДАТОК А. ПРОГРАМНИЙ КОД

Повний програмний код роботи розміщено в репозиторії: https://github.com/kqjqxb/Software-Security, коміт 4bb9629. Файли роботи – у каталозі `lab2/lab02_access_control`: `mypolicy.py`, `detector.py`, `storage.py`, `demo.py`, `tests/test_variant.py`.